

GROUP 5:

Axel Hedman 791490

David Enberg 792143

Markus Andersson 791759

Alfred Backlund 900401

PROJECT PLAN

1. SCOPE OF WORK

1.1. GENERAL

Our plan is to build a tower defence game with inspiration from classical tower defence games. We plan to build a tower defence game inspired by the likes of Bloons Tower Defence and Kingdom Rush. The game will consist of multiple features like:

- Maps that are read from a file.
- Towers:
 - Standard tower that damages one enemy at a time.
 - Freeze tower that freezes enemies during a short period of time.
 - Bomb tower with an area of effect damage.
 - Poison tower with damage over time.
- Enemies:
 - Basic enemy
 - First round hp: 1.
 - Standard speed.
 - Immune/stealthy enemy
 - Not affected by AOE damage and damage over time.
 - Squishy and fast.
 - BOSS enemy that splits into smaller enemies when killed.
 - As levels increase the boss splits into more enemies.

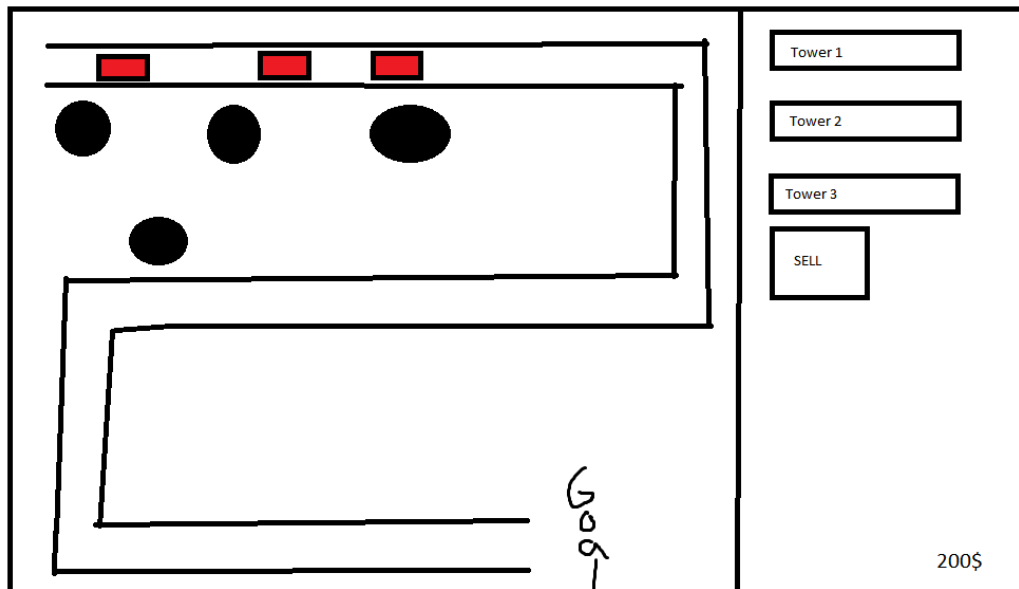
The towers are bought with money that the player gets from killing enemies. After buying a tower the player will be able to upgrade it for more damage and faster wave clear. The enemies' hp increases over time and wave sizes get bigger as the game progresses. The game will include 7 different levels with increasing difficulty.

1.2. FUNCTIONALITY

The game will be played with the mouse. At least a rudimentary user interface will be implemented to show resources, waves, number of enemies, health, shop, and score. There will be sound effects when towers attack and enemies die. The purpose of the program is to entertain. When starting the program, the player will be redirected to the main menu with three options:

- Level editor
- Level selector/play game
- Options

When playing the game all interactions will be done using the mouse, for example buying new tower, upgrading old ones or selling towers.



Rough sketch of the user interface.

2. HIGH-LEVEL STRUCTURE

2.1. PARENT CLASSES

The program will consist of at least five a parent classes. Towers, enemies, maps, game engine and graphics all require a parent class of their own. The graphics parent class will be the class that every other inherits from. The number of parent classes is still unsure and there will most certainly be changes as the development progresses.

2.2. CHILD CLASSES

The diagram below represents the relationships between parent classes and child classes. As there are still uncertainties on how the game engine and map will be implemented, we cannot foresee which child classes will be needed in those areas.

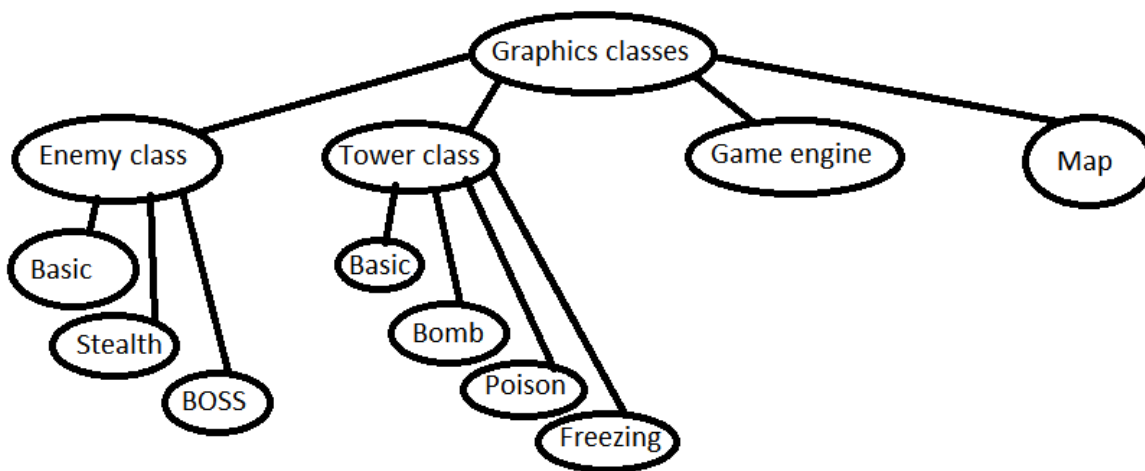


Diagram of planned class relationships.

3. EXTERNAL LIBRARIES

When implementing this program, the SFML framework will be our main tool. SFML allows us to efficiently and seamlessly add features like sound effects and visuals. This will require every group member to individually study the SFML framework to be able to implement it in the project.

We started the project trying to implement the Qt library but after some testing and comparing we figured that SFML is better suited for our implementation. The SFML library will be used for all visual and auditory aspects of the project, from rendering graphics to implementing sound effects and implementing hitboxes.

Most likely, we will add more external libraries during the project. Right now, it is difficult to predict everything, but SFML will probably take some time to get used to. Therefore, it is good to start with it

4. DIVISION OF WORK

Our group will meet at least twice a week. During these meetings we will discuss the workflow and divide our work into personal tasks. These weekly meetings allow us to ensure that work gets done progressively.

At this point there are five main classes that requires implementation. Every group member will have their own area of expertise, but everyone will also be assigned a secondary area to supervise. This will ensure that every chunk of code that gets added to the project has been looked at by at least two of the group members.

Areas of Responsibility:

- Graphics
 - Primary: David
 - Secondary: Axel
- Enemies & Towers
 - Primary: Alfred
 - Secondary: Markus
- Game engine
 - Primary: Markus
 - Secondary: David
- Map
 - Primary: Axel
 - Secondary: Alfred

5. SCHEDULE

Our goal is of course to keep all the deadlines by staying ahead of the schedule. As earlier mentioned, our work schedule will consist of at least two meetings a week where we will assist each other and plan the forthcoming work. We plan on finishing our demo 2.12 and the demo will include:

- The minimum requirements for a passing grade.
 - A functioning tower defence game
 - Three different types of towers
 - Three different types of enemies
 - Five different levels with increasing difficulty
 - The user can interact with the program using the mouse

- User interface that shows basic information
- As many additional features as possible if the program works.

We will use the time between the demo submission and the final commit to add further features to increase the project grade. So, our main goal right now is to implement a working program that gets us through the minimum requirements. After that it's simple to continue building our final product.

As for the division of work, the schedule will be easier to lay out once we know more about how SFML works and how we need to implement it in our game. This will make it possible to define checkpoints for our work, enabling a more streamlined workflow.

Timeline

25.11

- Map generation from file
- Basic UI (Place/play, show resources)
- Projectiles are working (enemies can get killed)

30.11

- Money and upgrades are working
- 5 levels
- Added sound effects
- Level editor

2.12

- Demo version

2-9.12

- Testing